Git 手册

基础概念与术语

Git 在本地跟踪变更。GitHub 在线托管仓库并提供协作工具。

- 工作目录：你正在编辑的文件。
- 暂存区：已选中、用于下一次提交的变更。
- 提交：带有说明信息和唯一 ID 的已保存快照。
- 分支：指向某个提交的名字；你在其上提交时它会前移。
- HEAD：通常指向你当前的分支，也就是它的最新提交。
- 远程：到另一个仓库的连接，通常命名为 origin。
- `main` 与 `origin/main`：你的本地分支 与 你对远端分支的本地记录，后者由 fetch 和成功的 push 来更新。

常用命令

命令	用途
`git init -b main`	在当前文件夹中初始化一个仓库，并创建名为 main 的分支。
`git clone <url>`	将已有仓库下载到一个新文件夹。
`git remote add origin <url>`	将本地仓库连接到一个名为 origin 的远程仓库。
`git status`	显示当前分支以及已暂存、未暂存或未跟踪的文件。
`git diff`	显示对已跟踪文件的未暂存变更。
`git diff --staged`	显示已选中、用于下一次提交的变更。
`git add <file>`	将该文件当前的变更加入暂存区。之后的编辑需要再次 add。
`git commit -m "message"`	在本地保存已暂存的变更。
`git log --oneline`	显示精简的提交历史。
`git branch`	列出本地分支。
`git switch -c feature`	创建并切换到一个名为 feature 的分支。你可以选择不同的分支名称。
`git switch main`	切换到已有的 main 分支。
`git merge feature`	将名为 feature 的分支合并到当前分支。
`git rebase main`	将当前分支的提交重放到 main 之上。
`git fetch origin`	下载更新，但不将其合并到当前分支。

命令	用途
`git pull`	获取并整合远端更新；具体方式由参数和配置控制。
`git push`	上传本地提交并更新对应的远端分支。
`git push -u origin feature`	推送该分支，并为其设置上游，以便后续 pull 和 push。
`git stash` / `git stash pop`	搁置未提交的已跟踪变更 / 重新应用最近一次 stash，并在成功时将其移除。
`git restore --staged <file>`	取消暂存变更，同时保留你的编辑。
`git restore <file>`	用已暂存的版本替换工作区文件内容，丢弃未暂存的编辑。
`git revert <commit>`	创建一个新提交来撤销之前的某个提交。
`git reset --soft <commit>`	将当前分支回退，同时让后续变更保持暂存。

重要区别

对比项	区别
提交 与 推送	提交保存在本地。推送则把提交分享到远程仓库。
init 与 clone	init 开启本地历史。clone 复制已有历史并配置远程。
合并 与 变基	合并保留已有提交，并可能新增一个合并提交。变基以新 ID 重放提交，在新基底上形成线性序列。
快进 与 合并提交	快进只前进分支指针。合并提交连接两条历史，并且有多个父提交。
fetch 与 pull	fetch 更新远程跟踪信息。pull 还会把更新整合进你的当前分支。
revert 与 reset	revert 在历史中记录一次撤销。reset 移动分支指针；其模式控制暂存变更和文件如何处理。
pull 与 pull request	pull 更新本地分支。pull request 提议将一个分支整合到 GitHub 上的另一个分支。

显式指定 pull 的行为：

- `git pull --ff-only`：仅当你的分支能够前进、而无需合并分叉的历史时，才进行更新。
- `git pull --rebase`：将你仅在本地的提交重放到获取到的分支之上。
- `git pull --no-rebase`：用一次合并来整合获取到的分支。

不要对其他人正在使用的已发布提交进行变基或重置。reset --soft 保持
变更处于暂存状态；默认的 --mixed 保持变更未暂存；--hard 还会覆盖已跟踪的
工作区文件，并可能删除造成阻碍的未跟踪文件。

Pull Request（PR）是 GitHub 上的评审流程，不是 Git 命令。打开一个 PR 并不会
合并任何内容。一个 PR 可以包含讨论、自动化检查以及多个提交。GitHub 可以根据
仓库设置，用合并提交、squash 合并或 rebase 合并来整合它。

当 Git 无法自动协调变更时，就会发生冲突。把相关文件编辑成期望的
最终状态，删除冲突标记，暂存解决方案，然后完成合并或变基。

参考：merge、rebase、pull、reset 以及 GitHub 的 pull request。

自己动手试试

你可以从这里下载 Git：https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

按顺序完成这些练习，一次一个代码块。使用 Linux/macOS 上的 Bash，或 Windows 上的
Git Bash，Git 版本需 2.28 或更高。所有项目内容都将是纯文本文件。

在同一个终端中运行命令。除非另有说明，否则始终停留在 git-playground 目录。如果
某个命令意外失败，先修复它再继续。预期会失败的步骤都已标注。如果 Git 打开分页器，
按 q 退出。

Shell 提示：echo 写入文本，> 创建或覆盖文件，>> 追加内容，cat 显示文件内容，ls
列出文件。这些都是 shell 命令，不是 Git 命令。

1. 创建一个空的 GitHub 仓库

1. 登录 GitHub，创建一个名为 git-playground 的仓库。这里的“项目”指的是
   一个仓库，而不是 GitHub Projects 看板。
2. 选择私有或公开可见性。
3. 保持 README、.gitignore 和 license 初始化处于关闭状态，使仓库为空。
4. 复制它的 HTTPS URL：https://github.com/YOUR_USERNAME/git-playground.git

把下面的 YOUR_USERNAME 替换成你的真实用户名。对于 HTTPS 推送，在提示输入密码时
使用你配置的凭据管理器或个人访问令牌；你的 GitHub 账户密码
是无法使用的。参见 GitHub authentication。

2. 创建文件夹并连接到 GitHub

从一个父目录开始，你希望在该目录中保存此练习：

```
mkdir git-playground
cd git-playground
git init -b main

git config user.name "Your Name"
git config user.email "YOUR_EMAIL"

# 这些命令创建文件并写入内容。
```

```
echo "My Git practice notes" > notes.txt
echo "Hello world" > greeting.txt
echo "color=blue" > settings.txt

# 检查当前工作区
git status
# 将文件暂存以准备提交
git add notes.txt greeting.txt settings.txt
git diff --staged
# 提交你的变更
git commit -m "Create practice files"

# 将本地 git 项目与线上仓库关联
git remote add origin https://github.com/YOUR_USERNAME/git-playground.git
git remote -v
# 将本地提交推送到线上仓库
git push -u origin main
```

把姓名和邮箱替换成你自己的信息；GitHub 提供的 no-reply 邮箱也可以。这些设置只作用于当前仓库，用于标识提交；它们并不会认证你的身份。

检查：刷新 GitHub，三个文本文件都应出现在 main 上。

更多文档：Adding local code to GitHub。

3. 查看、暂存、提交并推送一次编辑

```
echo "I can inspect and save changes." >> notes.txt
git status
git diff
git add notes.txt
git diff
git diff --staged
git commit -m "Add a practice note"
git log --oneline -3
git push
```

检查：暂存前，git diff 显示你的编辑。暂存后，git diff 为空，而 git diff --staged 显示该编辑。提交后，工作区是干净的。只有 push 才会更新 GitHub。

4. 创建分支并合并它

```
git switch -c feature-list
echo "Practice branching" > tasks.txt
git add tasks.txt
git commit -m "Add practice tasks"
git branch

git switch main
```

```
ls
git merge --no-ff feature-list -m "Merge practice tasks"
cat tasks.txt
git log --graph --oneline --all -8
```

检查：切换到 main 时 tasks.txt 消失，合并后又出现。--no-ff
会特意创建一个合并提交，尽管这个例子本来可以快进。

5. 触发并解决合并冲突

在两个分支上以不同方式修改同一原始行：

```
git switch -c friendly-greeting
echo "Hello friends" > greeting.txt
git add greeting.txt
git commit -m "Use a friendly greeting"

git switch main
echo "Hello team" > greeting.txt
git add greeting.txt
git commit -m "Use a team greeting"

git merge friendly-greeting
```

预期失败：Git 报告内容冲突。检查它：

```
git status
cat greeting.txt
```

使用默认冲突样式时，文件中包含：

```
<<<<<<< HEAD
Hello team
=======
Hello friends
>>>>>>> friendly-greeting
```

上半部分是当前分支的版本；下半部分是传入分支的版本。其他冲突
样式也可能显示原始行。

先练习取消合并，然后再次触发它：

```
git merge --abort
cat greeting.txt
```

```
git merge friendly-greeting
```

再次预期失败：冲突再次出现。通过选择最终句子来解决它。覆盖这个
单行文件也会移除所有冲突标记：

```
echo "Hello team and friends" > greeting.txt
git add greeting.txt
git commit -m "Resolve greeting conflict"
git status
```

检查：文件中包含你合并后的句子，工作区干净，合并完成。

6. 触发并解决变基冲突

创建一个修改颜色的分支，然后在 main 上修改同一行：

```
git switch -c color-update
echo "color=green" > settings.txt
git add settings.txt
git commit -m "Choose green"
git log --oneline -1

git switch main
echo "color=red" > settings.txt
git add settings.txt
git commit -m "Choose red"

git switch color-update
git rebase main
```

预期失败：Git 无法自动把“Choose green”重放到“Choose red”之上。与合并不同，
该操作会重写 color-update 上的提交；它不会移动 main。

检查、取消并重试：

```
cat settings.txt
git rebase --abort
cat settings.txt
git rebase main
```

再次预期失败：取消后，文件是绿色的；重试会再次产生冲突。用第三种颜色
来解决它：

```
echo "color=purple" > settings.txt
git add settings.txt
```

```
git -c core.editor=true rebase --continue
git log --oneline -3
```

-c core.editor=true 表示接受现有提交信息，而不为这条命令打开编辑器。
在变基过程中，冲突的 HEAD 侧代表新基底加上已重放的提交；
另一侧则是正在被重放的提交。

检查：“Choose green”现在有了不同的提交 ID，并且排在“Choose red”之后。它的信息保持不变，尽管你把内容解决成了 purple。

把结果合并进 main：

```
git switch main
git merge --ff-only color-update
cat settings.txt
git push
```

检查：结果是 color=purple。这次合并前移了 main，且没有新增合并提交。

7. 打开并合并一个 Pull Request

```
git switch -c pr-notes
echo "This file arrived through a pull request." > review.txt
git add review.txt
git commit -m "Add review notes"
git push -u origin pr-notes
```

在 GitHub 上：

1. 打开仓库的 Pull requests 标签页，选择 New pull request。
2. 把 base 设为 main，compare 设为 pr-notes。
3. 检查变更，输入标题，然后创建 PR。
4. 打开 Files changed 检查提议的新增内容。
5. 在合并选项中选择 Create a merge commit，然后合并并确认。这样可以把该分支的原始提交保留在本练习的 main 中。

检查：GitHub 的 main 包含 review.txt。你的本地 main 还没有更新。

8. 观察 fetch 与 pull 的区别

```
git switch main
git fetch origin
git log --oneline main..origin/main
git diff main origin/main
ls
```

检查：Git 知道了远端提交，但 review.txt 在你的工作文件夹中仍然不存在。
main..origin/main 范围表示从 origin/main 可达、但从 main 不可达的提交。

```
git pull --ff-only
cat review.txt
git branch -d pr-notes
```

检查：该文件现在出现了。删除已合并的本地分支仍会保留其在历史中的提交；此命令
不会删除 GitHub 上的分支。

9. 克隆第二份副本并模拟队友

创建一个包含独立克隆的兄弟文件夹：

```
cd ..
git clone https://github.com/YOUR_USERNAME/git-playground.git git-
playground-peer
cd git-playground-peer
git config user.name "Your Name"
git config user.email "YOUR_EMAIL"

echo "A teammate added this note." > team.txt
git add team.txt
git commit -m "Add teammate note"
git push
```

回到你的第一份副本，在不 pull 的情况下再提交一次变更：

```
cd ../git-playground
echo "My simultaneous local change." >> notes.txt
git add notes.txt
git commit -m "Add local work"
git push
```

预期失败：推送被拒绝，因为 GitHub 上有你的本地分支缺失的提交。这是
历史分叉，还不是文件内容冲突。

```
git fetch origin
git log --oneline --left-right main...origin/main
git pull --ff-only
```

预期失败：快进无法协调分叉的分支。在日志中，< 标记仅在本地的
提交，> 标记仅在远端的提交。

把未发布的本地工作重放到远端工作之上：

```
git pull --rebase
git log --oneline -4
cat team.txt
git push
```

检查：两处变更都得以保留。由于提交修改的是不同文件，因此没有内容冲突。

10. 对比 pull 与合并

从第二份副本再制造一次远端变更：

```
cd ../git-playground-peer
git pull --ff-only
echo "Another teammate change." >> team.txt
git add team.txt
git commit -m "Extend teammate note"
git push

cd ../git-playground
echo "Another local change." >> notes.txt
git add notes.txt
git commit -m "Extend local notes"
git pull --no-rebase --no-edit
git log --graph --oneline -6
git push
```

检查：这次 Git 新增了一个连接两条历史的合并提交。已有本地提交保留
它的 ID。--no-edit 接受 Git 生成的合并信息。

11. 暂存未完成的工作

```
echo "An unfinished idea." >> notes.txt
git diff
git stash push -m "Unfinished notes"
git status
git stash list
cat notes.txt

git switch feature-list
git switch main
git stash pop
git diff
git add notes.txt
git commit -m "Finish the stashed idea"
```

检查：这个想法在 stash 时消失，pop 后回来。普通 stash 不包含未跟踪文件；使用 git stash push -u 可包含它们。如果目标发生了变化，pop 可能冲突；此时 stash 会被保留。参见 stash。

12. 用 revert 撤销已发布的提交

```
echo "This file was a mistake." > mistake.txt
git add mistake.txt
git commit -m "Add an unwanted file"
git push

git revert --no-edit HEAD
git log --oneline -3
ls
git push
```

检查：mistake.txt 消失。原始提交仍保留在历史中，其后跟着一个新提交来逆转它。HEAD 选择了 revert 时最新的提交。

13. 用 reset 重做未发布的提交

使用一个全新的本地分支：

```
git switch -c reset-practice
echo "Draft wording" > draft.txt
git add draft.txt
git commit -m "Temporary draft"

git reset --soft HEAD~1
git status
git diff --staged
git log --oneline -2
```

检查：临时提交不再处于分支顶端，但 draft.txt 仍然存在，且其新增处于暂存状态。HEAD~1 表示当前提交的第一个父提交。

改进草稿并再次提交：

```
echo "Final wording" > draft.txt
git add draft.txt
git commit -m "Add the final draft"
git switch main
git merge --ff-only reset-practice
```

检查：新提交包含最终措辞。临时提交从未被推送。

14. 取消暂存一次编辑，然后丢弃它

```
echo "An edit I do not want." >> notes.txt
git add notes.txt
git restore --staged notes.txt
git status
git diff

git restore notes.txt
git status
```

检查：取消暂存会在文件中保留该编辑。之后 restore 会丢弃它，因为已暂存的版本现在与最后一次提交一致。这里只丢弃练习中的编辑；Git 通常无法恢复从未提交或从未 stash 过的编辑。

15. 完成并检查结果

```
git push
git status
git log --graph --oneline --decorate --all -25
```

检查：你在 main 上，工作区干净，GitHub 上有最终文件。历史包含分支提交、合并提交、重放提交，以及一次显式的 revert。